

Nexa Secure Chat

Промпт на исправление: 7 проблем, блокирующих релиз

Подготовил: Senior QA Engineer · Дата: 2026-06-18 · Прод: <https://nexa-c.com>
Источник: ClaudeMemory.md (внутренняя память проекта).

КАК ПОЛЬЗОВАТЬСЯ ЭТИМ ДОКУМЕНТОМ

Это техническое задание для исполнителя (инженер или AI-агент). Каждая проблема описана по единому шаблону: Контекст → Что не так → Что сделать → Критерии приёмки (Definition of Done) → Файлы → Верификация. Не чинить вслепую: сначала сверяться с реальным кодом, премиса может разойтись с фактами. Ничего не деплоить, пока не пройдены критерии приёмки и тесты.

ЖЁСТКИЕ ПРАВИЛА ПРОЕКТА (не нарушать)

- ClaudeMemory.md НЕ деплоится на сервер и НЕ попадает в публичный репозиторий.
- В публичный репозиторий — без AI-авторства в коммитах и без security-чувствительного содержимого.
- Любое UI-изменение оптимизируется под все устройства и ОС.
- Не читать/не логировать секреты: пароли, хеши, ключи, токены, .env.
- Frontend: rsync в /home/ubuntu/nexa/frontend/web/dist/ (НЕ nexa-dist — устаревшая копия).
Backend: пересборка Docker-образа.

СВОДКА ПРИОРИТЕТОВ

BLOCKER — релиз нельзя выпускать, пока не закрыто. MAJOR — функционал/доверие страдает, закрыть до публичного анонса. TEST — реализация вероятно корректна, но нет регрессионного покрытия; нужен автотест, фиксирующий поведение.

#	Проблема	Severity
1	QR refresh-токен plaintext в БД (фикс не закоммичен)	BLOCKER
2	Web анти-скриншот: deterrent, а не блокировка	BLOCKER (claim)
3	E2EE только Phase 1 (нельзя заявлять 'fully E2EE')	BLOCKER (claim)
4	Presence = polling 30s, только активный чат	MAJOR
5	WebAuthn – заглушка	MAJOR
6	Login risk scoring – alpha, ложные блокировки	MAJOR
7	Single-session + QR max-2: нет регресс-теста	TEST

Детальные задания

#1. QR refresh-токен в plaintext — фикс живёт только в рабочем дереве

BLOCKER

КОНТЕКСТ

Столбец `qr_sessions.refresh_token_raw` (`db/models.py:59`, тип `Text`) хранил PLAINTEXT refresh-токен для передачи на спаренное QR-устройство и не очищался после `qr_poll`. Сделан фикс `consume_qr_refresh(token)` — одноразовое чтение + очистка, в in-memory `SessionStore` и `Postgres repository`, вызывается в `qr_poll` после `_set_refresh_cookie`.

ЧТО НЕ ТАК

Фикс НЕ закоммичен и НЕ задеплоен — существует только в рабочем дереве. Один `git checkout/stash drop` — и уязвимость возвращается. Нет регрессионного теста, доказывающего, что plaintext не остаётся в БД после первого чтения.

ЧТО СДЕЛАТЬ

- Сверить текущее состояние рабочего дерева: убедиться, что `consume_qr_refresh` реализован в обоих хранилищах и вызывается ровно один раз в `qr_poll`.
- Добавить интеграционный тест: после `qr_poll` повторное чтение `refresh_token_raw` возвращает `None/пусто`; токен в БД отсутствует.
- Закоммитить (без AI-авторства, без секретов в сообщении) и задеплоить backend (пересборка образа `auth-service`).

КРИТЕРИИ ПРИЁМКИ

- Запрос к БД после успешного `qr_poll`: `refresh_token_raw IS NULL` для этой сессии.
- Повторный `qr_poll` тем же кодом не выдаёт refresh повторно (одноразовость).
- Тест падает, если откатить фикс (защита от регресса доказана).
- Изменение схемы БД не требуется; `py_compile/импорт` чистые.

ФАЙЛЫ

```
backend/auth-service: db/models.py, repository.py (SessionStore + Postgres),
session_routes.py (qr_poll)
tests/integration/ - новый тест test_qr_refresh_consume.py
```

ВЕРИФИКАЦИЯ

```
AUTH_DATABASE_URL= REDIS_URL= pytest tests/integration/ -m integration -k qr -v
docker compose build auth-service && docker compose up -d --no-deps auth-service
docker compose ps # healthy
```

#2. Web анти-скриншот — deterrent, не блокировка (false marketing claim)

BLOCKER

КОНТЕКСТ

Браузерные API физически не позволяют блокировать OS-level скриншоты (`PrintScreen`, `Win+Shift+S`, `Cmd+Shift+4` перехватываются ОС до браузера). `privacySeal.ts` даёт `blackout`, `Keyboard Lock request` и `focus-loss protection` — это deterrent, лучшее из возможного. Натив (`Android FLAG_SECURE`, `Tauri set_content_protection`) — полноценная блокировка.

ЧТО НЕ ТАК

Если в UI, на лендинге или в текстах есть формулировка вроде «полная блокировка скриншотов в

браузере» — это ложное утверждение и репутационный/юридический риск.

ЧТО СДЕЛАТЬ

- Провести аудит всех пользовательских текстов (лендинг, settings, тултипы, маркетинг) на тему скриншотов.
- Заменить формулировки на честные: 'защита от скриншотов на нативных приложениях; в браузере — максимальное противодействие, но ОС-уровень не гарантирован'.
- Опционально: бейдж/иконка уровня защиты в зависимости от платформы (web vs native).

КРИТЕРИИ ПРИЁМКИ

- Нигде в web-UI не обещается «полная/гарантированная» блокировка скриншотов в браузере.
- Текст различает native (блокировка) и web (deterrent).
- Тексты согласованы на всех языках локализации.

ФАЙЛЫ

frontend/web: лендинг/маркетинг-копи, Settings (privacy секция),
i18n/локали, privacySeal.ts (комментарии-документация)

ВЕРИФИКАЦИЯ

```
grep -ri 'скриншот\|screenshot\|block' frontend/web/src # ревью формулировок  
npx tsc --noEmit && npm run build
```

#3. E2EE только Phase 1 — нельзя заявлять 'fully E2EE'

BLOCKER

КОНТЕКСТ

e2ee.ts + e2ee_envelope (client-side) разработаны, но НЕ все сообщения шифруются end-to-end: серверные ключи ещё используются на части маршрутов. chat-service хранит сообщения в БД с field encryption, но это не E2EE в полном смысле.

ЧТО НЕ ТАК

Маркетинговое утверждение 'fully E2EE' не соответствует коду. Заявлять его до Phase 2 нельзя.

ЧТО СДЕЛАТЬ (минимум для снятия блокаера)

- Аудит текстов: убрать/исправить любые 'fully/end-to-end encrypted by default'.
- Зафиксировать честный статус: какие маршруты E2EE, какие — field-encryption на сервере.
- Создать/обновить тех-задачу на E2EE Phase 2 (шифровать все сообщения на клиенте, на сервере только зашифрованные blob-ы).

КРИТЕРИИ ПРИЁМКИ

- UI/маркетинг не утверждают полное E2EE там, где его нет.
- Есть письменная матрица: маршрут → тип шифрования (E2EE / server-side field).

ФАЙЛЫ

frontend/web: маркетинг/лендинг, security-описания
backend/chat-service, frontend/web/src e2ee.ts/e2ee_envelope (для матрицы статуса)

ВЕРИФИКАЦИЯ

Ревью матрицы шифрования + grep по 'E2EE'/'end-to-end' в текстах UI.

#4. Presence — polling 30s, обновляется только активный чат

МАJOR

КОНТЕКСТ

ChatContext.tsx делает 30s poll fetchPublicProfile(activePeerId) только для активного чата + visibilitychange listener. Sidebar у остальных чатов не обновляется.

ЧТО НЕ ТАК

Это polling с задержкой до 30с, а не real-time presence. Заявлять 'real-time presence' нельзя. Sidebar показывает устаревший онлайн-статус.

ЧТО СДЕЛАТЬ

- ws-gateway/app/ws/handler.py: в finally-блоке при дисконнекте broadcast presence.offline к peers.
- presence-service: публиковать presence.update всем подписчикам peer, не только subject.
- Frontend: подписаться на presence-события через WS и обновлять статус в sidebar, а не только в активном чате.

КРИТЕРИИ ПРИЁМКИ

- Закрытие/обрыв сессии одним юзером отражается у всех его контактов < 2с (а не 30с).
- Sidebar обновляет онлайн-статус без открытия чата.
- Нет регресса: 30s poll можно убрать или оставить как fallback.

ФАЙЛЫ

```
backend: ws-gateway/app/ws/handler.py, presence-service/*
frontend/web/src: ChatContext.tsx, ChatSidebar
```

ВЕРИФИКАЦИЯ

```
2 клиента, разные аккаунты: А закрывает вкладку → В видит offline в sidebar < 2с.
pytest tests/integration -k presence -v
```

#5. WebAuthn — заглушка, не должна быть видна как рабочая опция

МАJOR

КОНТЕКСТ

webauthn_store.py и webauthn.ts существуют, но это stub. Реальная ceremony (challenge → attestation → assertion) не реализована.

ЧТО НЕ ТАК

Если в UI есть кнопка/опция WebAuthn (passkey), пользователь идёт по пути, который не работает — сломанный UX и иллюзия безопасности.

ЧТО СДЕЛАТЬ (минимум)

- Скрыть WebAuthn за feature-flag (по умолчанию off в production), пока ceremony не реализована.
- Либо реализовать полноценный flow: генерация challenge, верификация attestation, assertion.

КРИТЕРИИ ПРИЁМКИ

- В production пользователь не видит нерабочую опцию WebAuthn (если не реализовано).
- Если реализовано: успешные регистрация и вход через passkey на ≥2 платформах.

ФАЙЛЫ

```
backend/auth-service: webauthn_store.py
frontend/web/src: webauthn.ts + точки входа в UI, feature-flag
```

ВЕРИФИКАЦИЯ

Прод-сборка: опция скрыта/работает по флагу. `tsc --noEmit`, build OK.

#6. Login risk scoring — alpha, ложные блокировки при смене IP

MAJOR

КОНТЕКСТ

nexa_shared/security/login_risk.py — IP anomaly + device change scoring. Алгоритм базовый, threshold не настроен под прод.

ЧТО НЕ ТАК

Может ложно блокировать мобильных юзеров при смене Wi-Fi↔LTE (IP меняется в рамках сессии). False-positive убивает retention.

ЧТО СДЕЛАТЬ

- Не блокировать только по смене IP в рамках живой сессии; учитывать стабильные сигналы (device fingerprint, гео-дистанция/время).
- Вынести threshold в конфиг/env, задать консервативные значения для прода.
- Добавить мягкий путь (step-up: повторный 2FA) вместо жёсткой блокировки на средний риск.

КРИТЕРИИ ПРИЁМКИ

- Тест: один пользователь, смена IP Wi-Fi→LTE в активной сессии НЕ приводит к блокировке.
- Высокий риск (новый девайс + далёкое гео + ночь) → step-up, не глухой блок.
- Threshold конфигурируем, без хардкода.

ФАЙЛЫ

```
backend/shared/nexa_shared/security/login_risk.py
auth-service (точка вызова scoring), config/env
tests/unit/test_login_risk.py
```

ВЕРИФИКАЦИЯ

```
pytest tests/unit -k login_risk -v
```

#7. Single-session + QR max-2 инвариант — нет регресс-теста

TEST

КОНТЕКСТ

issue_tokens_for_user(revoke_others=True) по умолчанию — каждый логин убивает другие сессии. Исключение: qr_approve вызывает с revoke_others=False + инвариант max-2: keep {approver_sid, new_session.id}, revoke everything else.

ЧТО НЕ ТАК

Реализация вероятно корректна, но граничный кейс (3-я сессия через QR) не покрыт автотестом. Регресс здесь = тихий обход single-session policy.

ЧТО СДЕЛАТЬ

- Тест: обычный логин ревокает все прочие сессии (остаётся ровно 1).
- Тест: QR-approve оставляет ровно 2 (approver + новая), всё остальное ревокнуто.
- Тест: попытка завести 3-ю сессию через QR соблюдает инвариант max-2.

КРИТЕРИИ ПРИЁМКИ

- Три теста зелёные; при ломании инварианта в коде — падают.
- reuse-detection / family-revoke токенов покрыты хотя бы одним тестом.

ФАЙЛЫ

```
backend/auth-service: token_service.py, session_routes.py (qr_approve)
```

```
tests/integration/test_session_policy.py
```

ВЕРИФИКАЦИЯ

```
AUTH_DATABASE_URL= REDIS_URL= pytest tests/integration -m integration -k session -v
```

Общий Definition of Done для всего пакета

- Все BLOCKER закрыты; MAJOR закрыты до публичного анонса; TEST — автотесты в CI зелёные.
- `npm tsc --noEmit` чисто, `npm run build` успешен; `pytest tests/ -v` зелёный.
- Backend-изменения задеплоены через пересборку Docker-образа; контейнеры healthy.
- Frontend задеплоен в `/home/ubuntu/nexa/frontend/web/dist/` (HE nexa-dist); `curl https://nexa-c.com` → 200, bundle hash совпадает.
- Коммиты без AI-авторства и без security-чувствительного содержимого; ClaudeMemory.md не задеплоен и не в публичном репо.
- ClaudeMemory.md обновлён: что сделано, что задеплоено, дата.